

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-a320695728e8cbdd5.jsonl`  
- SHA-256: `0315218e2d69ded4e49f3060c958a61b624c072c9f180fe2e37c2646f2ed71fe`  
- Source modified: `2026-07-04T00:46:48+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `subagents`  
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-04T00:44:13.410Z)

Code-review finder. Target: PR #458 of badminton-highlight-indexer. Diff: `cd C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer && gh pr diff 458`. PR branch = `origin/codex/a7-a9-quality-chain` (read via `git show origin/codex/a7-a9-quality-chain:<path>`; checkout is on master). Files also at scratchpad/pr458_fusion_golden.py, pr458_fusion_audio.py.

ANGLE B ? removed/replaced behavior. For every DELETED or REPLACED line, name the invariant it enforced and check the new code re-establishes it. Focus:

1. In fusion_golden.run() the OLD resume guard was `if not fresh and os.path.isfile(out): skip`. The NEW code splits into existing/existing_full branches. Did the change LOSE the simple "file exists ? skip" for any case? Specifically: a video with an existing FULL-fusion JSON ? old code skipped it; does new code still skip it in ALL invocation modes (default, --missing-fusion-only)? And a windowing-only JSON ? old code SKIPPED it (bug: kept stale), new code re-extracts. Confirm the new behavior is the intended fix and doesn't accidentally re-extract full-fusion files (wasting a GPU pass).
2. tests/test_ablation.py: the litmus test was rewritten from asserting exactly n=6 to `len(videos) in {6,15}`. Did this WEAKEN the test such that a corpus with a WRONG count (e.g. 11 mixed-schema, the documented bug) now passes? Read `\_golden\_present` and `load\_videos` ? does the test still guard the n=11 mixed-schema bug it mentions?
3. test_fusion_compare.py / test_audio_features.py: added `"video\_path": \_\_file\_\_` and a `Foo.MP4` file to existing tests ? was this to satisfy a NEW os.path.isfile(video) guard? Confirm the guard doesn't break any pre-existing caller that passed a spec without a real video file (grep callers of extract_video / augment in non-test code).
4. The `--max-window-duration 0 --inactivity-end 0` in the doc's A8 command vs prior ? any behavior claim removed?

Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}]. [] if none. Final message = only the JSON array.

2. assistant (2026-07-04T00:44:15.459Z)

I'll analyze PR #458 for removed/replaced behavior that might break invariants. Let me start by getting the diff and examining the key files.

3. user (2026-07-04T00:44:17.060Z)

```
diff --git a/SESSION_HANDOFF.md b/SESSION_HANDOFF.md  
index 31ea4c14..a50cc8a1 100644  
--- a/SESSION_HANDOFF.md  
+++ b/SESSION_HANDOFF.md  
@@ -2,49 +2,51 @@
```

You are here

-A6 is implemented and open for review as PR #455:

-`https://github.com/Khelsutra/badminton-highlight-indexer/pull/455`.
+A6 PR #455 is merged. This checkout is on `codex/a7-a9-quality-chain`, fast-forwarded to
+`origin/master` at `c5fc27c`, with tracked edits in:

-The branch is `codex/a6-ablation-cli`, rebased onto `origin/master` after A5
-merged as `c8eec7c` / PR #454. It fixes the `python -m backend.eval.ablation`
-circular import by making `ablation.export\_pdf` a lazy wrapper, adds a
-module-entripoint regression test, and records the n=15 R2/tolF1 contrast in
-`docs/ALPHA\_LAUNCH\_READINESS.md`.
+- `backend/eval/fusion\_golden.py`
+- `backend/eval/fusion\_audio.py`
+- `tests/test\_fusion\_compare.py`
+- `tests/test\_audio\_features.py`
+- `docs/ALPHA\_LAUNCH\_READINESS.md`
+- `SESSION\_HANDOFF.md`

-Validation completed locally on 2026-07-03:
-`ruff check .`, `python -m mypy backend training`, and
-`python run\_all\_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`
-all passed. Full suite result: 1609 passed, 6 skipped, 85.65% coverage.
+A7/A8/A9 have been run locally against the 15-video corpus, and draft PR #458 is open:
+`https://github.com/Khelsutra/badminton-highlight-indexer/pull/458`. The generated
+feature/model artifacts live under ignored `output/` and `artifacts/` paths and should not
+be committed.

Next steps / open threads

-1. Review/merge PR #455 for A6.
-2. A7 is the next indexer-owned quality item: regenerate or upconvert
- full-fusion feature JSONs for the newer nine golden videos so fusion/person
- /audio analyses can run on all 15 videos. GPU is approved on this machine.
-3. A8 follows A7: rerun `fusion\_compare`, group ablation, and served contrast;
- explicitly promote or reject fusion/person/Gate-A served-default decisions.
-4. A3 is owned by another agent; do not duplicate that collector/vault work.
+1. Address review comments on draft PR #458; convert to ready only after owner/reviewer agrees
+ the A9 TemporalMaxer scope question is settled.
+2. Decide whether A9 is considered complete with the current Gen-0 non-serving shadow
+ bundle or whether TemporalMaxer needs a follow-up scorer-swap PR.
+3. Do not touch A1; another agent owns it.
+4. A3 is in collector PR #62 and records all 15 source videos; the two Boxhill originals
+ were verified on `F:\`.

Ramp-up kit

- `docs/ALPHA\_LAUNCH\_READINESS.md`
-- `docs/data\_pipeline/GOLDEN\_VIDEOS.md`
-- `docs/R2\_EVAL\_METRIC\_DESIGN.md`
-- `backend/eval/ablation.py`
-- `backend/eval/ablation\_pdf.py`
-- `backend/eval/rally\_seg\_eval.py`
-- `tests/test\_ablation.py`
-- `output/human\_lovo\_manifest.json`
+- `backend/eval/fusion\_golden.py`
+- `backend/eval/fusion\_audio.py`
+- `training/gen0/harness.py`
+- `eval\_baselines/heuristic\_lovo\_n15.json`
+- `output/a7\_remaining\_default.out.log`
+- `output/a8\_audio\_augment.out.log`
+- `output/a6\_r2\_ablation.json`
+- `artifacts/rally\_tal\_gen0/0.1.1-n15-shadow.20260704/manifest.json`

Key decisions

-- R2/tolF1 is supported for the baseline-off to served milestone contrast on
- the 15-video corpus: `Delta tolF1 = +0.225`, CI `[+0.165, +0.283]`,

```

- sign `14/0`, `p = 0.000`.
-- This does not approve fusion/person/audio default changes. Current full-fusion
- ablation still runs on only six videos because the newer nine do not yet have
- full-fusion feature schema.
-- Branching discipline remains important: always fetch/pull first, branch from
- `origin/master`, stage explicit paths, and verify `origin/master..HEAD` before
- pushing.
+- A7: full-fusion schema and audio features are present locally for all 15 videos.
+- A8: person fusion and audio are suggestive but not statistically clean; do not promote
+ either as a default. Gate A regresses under served windowing; keep it opt-in/analysis-only.
+- R2/cap/inactivity remains the strongest quality result on n15: F1 `0.237 -> 0.472`,
+ tolF1 `0.138 -> 0.364`, merge rate `0.638 -> 0.135`; net over-seg guard passes.
+- A9: Gen-0 LogReg/TAL n15 shadow bundle beats the heuristic mean (`0.551 > 0.4456`)
+ but is non-serving (`9/15` paired wins, `p=0.3036`; #172 and WASB C3 still block).
+- Branching discipline remains important: fetch/pull first, stage explicit paths, and
+ verify `origin/master..HEAD` contains only this branch's commits before pushing.
diff --git a/backend/eval/fusion_audio.py b/backend/eval/fusion_audio.py
index b5c65ec4..7dff0c94 100644
--- a/backend/eval/fusion_audio.py
+++ b/backend/eval/fusion_audio.py
@@ -17,14 +17,17 @@
     import json
     import logging
     import os
-from typing import Any, Dict, List
+from typing import Any, Dict, List, Optional, Sequence

    from backend.eval import experiment
    from backend.eval.fusion_compare import delta_stats
    from backend.eval.fusion_golden import (
        PERSON_FEATURE_NAMES,
        SHUTTLE_FEATURE_NAMES,
+    SOURCE_MANIFEST_ENV,
+    _derive_video_path,
+    _filter_specs,
+    attach_source_video_paths,
    )
    from backend.eval.golden_manifest import load_resolved_manifest
    from backend.pipeline.audio.hit_detector import detect_hits_in_video
@@ -33,9 +36,24 @@
    logger = logging.getLogger(__name__)

-def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
+def augment(
+    manifest_path: str,
+    features_dir: str,
+    k: float = 2.5,
+    source_manifest: Optional[str] = None,
+    only: Sequence[str] = (),
+) -> List[str]:
    """Add the 3 audio features to each existing ``<name>_golden_features.json`` window."""
    specs = load_resolved_manifest(manifest_path)
+    source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
    if source_manifest:
+        updated = attach_source_video_paths(specs, source_manifest)
+        print(
+            f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
+            f"from {source_manifest}",
+            flush=True,
+        )
    specs = _filter_specs(specs, only)
    written: List[str] = []
    for spec in specs:
        jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")

```

```

@@ -45,6 +63,10 @@ def augment(manifest_path: str, features_dir: str, k: float = 2.5) ->
List[str]:
    )
    continue
    video = _derive_video_path(spec)
+    if not os.path.isfile(video):
+        raise FileNotFoundError(
+            f"{spec.get('name')}: source video not found for audio augmentation: {video}"
+        )
    print(
        f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
        flush=True,
@@ -129,6 +151,21 @@ def main() -> None:
    ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
    ap.add_argument("--features-dir", default="output")
    ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
+    ap.add_argument(
+        "--source-manifest",
+        default=None,
+        help=(
+            "collector golden_source_videos.json; overrides video paths for source/proxy "
+            f"videos (or set {SOURCE_MANIFEST_ENV})"
+        ),
+    )
+    ap.add_argument(
+        "--only",
+        action="append",
+        default=[],
+        metavar="VIDEO_ID",
+        help="augment one manifest video by name; repeatable",
+    )
+    ap.add_argument(
+        "--augment",
+        action="store_true",
@@ -142,7 +179,13 @@ def main() -> None:
    args = ap.parse_args()

    if args.augment:
-        augment(args.manifest, args.features_dir, args.k)
+        augment(
+            args.manifest,
+            args.features_dir,
+            args.k,
+            source_manifest=args.source_manifest,
+            only=args.only,
+        )
    if args.compare:
        videos = load_videos_with_audio(args.features_dir)
        if len(videos) < 2:
diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index aee6d7e5..0666a0ca 100644
--- a/backend/eval/fusion_golden.py
+++ b/backend/eval/fusion_golden.py
@@ -21,7 +21,7 @@
import json
import os
import time
-from typing import Any, Dict, List, Optional
+from typing import Any, Dict, List, Optional, Sequence

from backend.eval import distill_local
from backend.eval.calibrate_wasb import load_golden_gts
write /dev/stdout: The pipe is being closed.

```

Let me get the complete diff and the files mentioned:

5. user (2026-07-04T00:44:23.363Z)

992 /tmp/pr458.diff

6. user (2026-07-04T00:44:25.957Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer.

7. assistant (2026-07-04T00:44:28.859Z)

The scratchpad files don't exist yet. Let me read the complete diff and the actual source files from the PR branch:

8. user (2026-07-04T00:44:30.130Z)

""Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.

Mirrors `distill\_local`'s CSV-driven flow but ADDS the ~5 person-cue features per shuttle candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini** and **no DB**, and writes one small JSON per video. The LOVO comparison (`fusion\_compare.py`) then runs 100% on CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is measured by the shipped experiment harness; person features enter only as feature columns the learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies them).

Run (on the GPU box):

```
python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json --out-dir output
```

Each output `output/<name>\_golden\_features.json` is the replayable feature substrate ? re-run `fusion\_compare` with different variants/thresholds forever without re-detecting on the GPU.

```
from __future__ import annotations
```

```
import argparse
import json
import os
import time
from typing import Any, Dict, List, Optional, Sequence
```

```
from backend.eval import distill_local
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.training import label_candidates
from backend.pipeline.detectors import fusion_features as ff
from backend.pipeline.detectors import rally_gate
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.utils.run_telemetry import RunTelemetry
```

The 5 fps/resolution-invariant shuttle features distill_local already computes.

```
SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
```

The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).

```
PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
```

```
SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
```

```
def is_full_fusion_feature_file(path: str) -> bool:
```

```
    """True when an existing golden feature JSON already has shuttle+person cue dicts.
```

Corpus-growth runs can leave leaner windowing-only JSONs with the same filename pattern. Those are useful for gts/windowing analysis, but they are not the replayable fusion substrate that ``fusion\_compare`` and ``ablation`` need.

"""

```
try:
    with open(path, encoding="utf-8") as f:
        data = json.load(f)
except (OSError, json.JSONDecodeError, TypeError):
    return False
candidates = data.get("candidates")
if not isinstance(candidates, list):
    return False
if not candidates:
    return True
return all(
    isinstance(candidate, dict)
    and "shuttle" in candidate
    and "person" in candidate
    for candidate in candidates
)
```

```
def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
    """Load ``video_id -> local_path`` from the collector golden-source manifest.
```

The collector records source/proxy ownership and full-file MD5s. Only entries marked ``present\_local`` with a concrete ``local\_path`` can drive local GPU extraction.

"""

```
if not source_manifest_path:
    return {}
with open(source_manifest_path, encoding="utf-8") as f:
    data = json.load(f)
entries = data.get("videos", data) if isinstance(data, dict) else data
if not isinstance(entries, list):
    raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
```

```
paths: Dict[str, str] = {}
for entry in entries:
    if not isinstance(entry, dict):
        continue
    video_id = entry.get("video_id") or entry.get("name")
    local_path = entry.get("local_path")
    if not video_id or not local_path:
        continue
    if entry.get("source_status", "present_local") != "present_local":
        continue
    paths[str(video_id)] = str(local_path)
return paths
```

```
def attach_source_video_paths(
    specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
) -> int:
    """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
```

Returns the number of specs updated. The collector source manifest is authoritative for local source/proxy video location and overrides derived/stale manifest paths.

"""

```
source_paths = load_source_video_paths(source_manifest_path)
updated = 0
for spec in specs:
    name = str(spec.get("name", ""))
```

```

        source_path = source_paths.get(name)
        if not source_path:
            continue
        if spec.get("video_path") == source_path:
            continue
        spec["video_path"] = source_path
        updated += 1
    return updated

def _filter_specs(
    specs: Sequence[Dict[str, Any]], only: Sequence[str]
) -> List[Dict[str, Any]]:
    if not only:
        return list(specs)
    wanted = list(dict.fromkeys(only))
    wanted_set = set(wanted)
    filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
    found = {str(spec.get("name")) for spec in filtered}
    missing = [name for name in wanted if name not in found]
    if missing:
        raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
    return filtered

def _derive_video_path(spec: Dict[str, Any]) -> str:
    """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
    golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming convention)."""
    if spec.get("video_path"):
        return str(spec["video_path"])
    labels = str(spec["labels"])
    if labels.endswith(".rallies.csv"):
        return labels[: -len(".rallies.csv")] + ".MP4"
    raise ValueError(
        f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"
    )

def extract_video(
    spec: Dict[str, Any],
    detector: Optional[Any] = None,
    frame_skip: int = 3,
    iou: float = 0.5,
    log_every_sec: float = 15.0,
    telemetry: Optional[Any] = None,
    use_gpu: bool = True,
    detector_model: Optional[str] = None,
    max_gpu_temp_c: Optional[float] = None,
) -> Dict[str, Any]:
    """One golden video -> a feature record (shuttle + person features per candidate window).

    Returns ``{name, fps, frame_width, candidates:[{start,end,shuttle{}},person{},label]},
    gts}``.

    `detector` (anything with ``detections_over_windows``) is injectable for GPU-free tests;
    when None a real `PersonDetector` is built (loads torchvision on first use).
    `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health snapshots
    from the detection progress callback so a hard kill leaves a forensic runlog.
    """
    fps, fw = float(spec["fps"]), float(spec["frame_width"])
    traj = str(spec["trajectory"])
    points = TrackNetRunner.parse_trajectory_csv(traj)
    intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
    gts = load_golden_gts(str(spec["labels"]))
    labels = label_candidates(intervals, gts, iou)
    # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic estimate

```

```

# (parity with the shuttle net-crossing gate) rather than hard-coding an orientation.
axis, net_px, _span = rally_gate.estimate_play_axis(points)
video_path = _derive_video_path(spec)
if not os.path.isfile(video_path):
    raise FileNotFoundError(
        f"{spec.get('name')}: source video not found for fusion extraction: {video_path}"
    )

if detector is None:
    from backend.pipeline.detectors.person_detector import PersonDetector

    indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
    if detector_model:
        indexing_cfg["detector_model"] = detector_model
    detector = PersonDetector({"indexing": indexing_cfg})

# Single-pass person detection over ALL candidate windows (open the video ONCE, read
# forward ? no per-window seek). Then compute features per window (pure, no GPU).
n = len(intervals)
win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
print(
    f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one pass...",
    flush=True,
)
t0 = time.monotonic()
state = {"last": t0}

def _progress(done: int, total: int) -> None:
    rec = None
    if telemetry is not None:
        rec = telemetry.snapshot(done, total, phase=spec["name"])
    if max_gpu_temp_c is not None and rec is not None:
        gpu = rec.get("gpu") or {}
        temp = gpu.get("temp_c")
        if temp is not None and float(temp) >= max_gpu_temp_c:
            raise RuntimeError(
                f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
                f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal shutdown"
            )
    now = time.monotonic()
    if now - state["last"] >= log_every_sec or done >= total:
        el = now - t0
        rate = done / el if el > 0 else 0.0
        eta = (total - done) / rate if rate > 0 else 0.0
        pct = 100 * done // total if total else 0
        print(
            f"[fusion_golden]   {spec['name']}: detect frame {done}/{total} ({pct}%) "
            f"| {el:.0f}s elapsed | ETA {eta:.0f}s",
            flush=True,
        )
        state["last"] = now

per_window = detector.detections_over_windows(
    video_path, win_frames, frame_skip, progress_cb=_progress
)

candidates: List[Dict[str, Any]] = []
for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
    det = per_window[i]
    det_fw = det.get("frame_width") or fw
    det_fh = det.get("frame_height") or 0.0
    # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-point
    # normalisation fusion_features uses (x/width, y/height).
    if axis == "x":
        net_norm = net_px / det_fw if det_fw > 0 else 0.5

```



```

else:
    net_norm = net_px / det_fh if det_fh > 0 else 0.5
    sf, ef = win_frames[i]
    shuttle_in = [p for p in points if sf <= p.frame <= ef]
    person = ff.compute_person_features(
        det["frames"],
        shuttle_in,
        det_fw,
        det_fh,
        court_polygon=None,
        net=(axis, net_norm),
    )
    candidates.append(
        {
            "start": float(s),
            "end": float(e),
            "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
            "person": {k: float(v) for k, v in person.items()},
            "label": int(labels[i]),
        }
    )
return {
    "name": spec["name"],
    "fps": fps,
    "frame_width": fw,
    "candidates": candidates,
    "gts": [[float(a), float(b)] for a, b in gts],
}

```

```

def run(
    manifest_path: str,
    out_dir: str,
    frame_skip: int = 3,
    iou: float = 0.5,
    fresh: bool = False,
    smallest_first: bool = False,
    source_manifest: Optional[str] = None,
    only: Sequence[str] = (),
    missing_fusion_only: bool = False,
    use_gpu: bool = True,
    detector_model: Optional[str] = None,
    max_gpu_temp_c: Optional[float] = None,
) -> List[str]:
    specs = load_resolved_manifest(manifest_path)
    source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
    if source_manifest:
        updated = attach_source_video_paths(specs, source_manifest)
        print(
            f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
            f"from {source_manifest}",
            flush=True,
        )
    specs = _filter_specs(specs, only)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
        # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
        row/frame).
        specs = sorted(
            specs,
            key=lambda s: (
                os.path.getsize(s["trajectory"])
                if os.path.isfile(s.get("trajectory", ""))
            )
        )

```

10. user (2026-07-04T00:44:35.852Z)

```

    "name": spec["name"],
    "fps": fps,
    "frame_width": fw,
    "candidates": candidates,
    "gts": [[float(a), float(b)] for a, b in gts],
}

def run(
    manifest_path: str,
    out_dir: str,
    frame_skip: int = 3,
    iou: float = 0.5,
    fresh: bool = False,
    smallest_first: bool = False,
    source_manifest: Optional[str] = None,
    only: Sequence[str] = (),
    missing_fusion_only: bool = False,
    use_gpu: bool = True,
    detector_model: Optional[str] = None,
    max_gpu_temp_c: Optional[float] = None,
) -> List[str]:
    specs = load_resolved_manifest(manifest_path)
    source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
    if source_manifest:
        updated = attach_source_video_paths(specs, source_manifest)
        print(
            f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
            f"from {source_manifest}",
            flush=True,
        )
    specs = _filter_specs(specs, only)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
        # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
        specs = sorted(
            specs,
            key=lambda s: (
                os.path.getsize(s["trajectory"])
                if os.path.isfile(s.get("trajectory", ""))
                else 0
            ),
        )
    os.makedirs(out_dir, exist_ok=True)
    # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so keep a tiny
    # rotating runlog of velocity + machine health for the post-mortem. Constructing it must
never
    # break extraction, so guard the build and fall back to no telemetry on any error.
    telem: Optional[RunTelemetry]
    try:
        telem = RunTelemetry(
            os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
        )
    except Exception: # noqa: BLE001 - telemetry wiring must never break the run it observes
        telem = None
    written: List[str] = []
    for vi, spec in enumerate(specs, 1):
        out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
        existing = os.path.isfile(out)
        existing_full = existing and is_full_fusion_feature_file(out)

```

```

# Resumable: skip a video whose full-fusion JSON already exists. A leaner
# windowing-only JSON is stale for A7, so re-extract it even without --fresh.
if missing_fusion_only and not fresh and existing_full:
    print(
        f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
        f"features already present -> {out}",
        flush=True,
    )
    written.append(out)
    continue
if not fresh and existing and existing_full:
    print(
        f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted "
        f"-> {out} (use --fresh to redo)",
        flush=True,
    )
    written.append(out)
    continue
if not fresh and existing and not existing_full:
    print(
        f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
        f"is not full-fusion schema; re-extracting...",
        flush=True,
    )
    print(
        f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
        flush=True,
    )
t0 = time.monotonic()
if telem is not None:
    telem.event("video_start", spec["name"])
try:
    data = extract_video(
        spec,
        frame_skip=frame_skip,
        iou=iou,
        telemetry=telem,
        use_gpu=use_gpu,
        detector_model=detector_model,
        max_gpu_temp_c=max_gpu_temp_c,
    )
except Exception as e: # noqa: BLE001 - record machine state at death, then re-raise
    if telem is not None:
        telem.event("error", spec["name"], err=str(e))
    raise
n_pos = sum(c["label"] for c in data["candidates"])
with open(out, "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)
if telem is not None:
    telem.event(
        "video_done",
        spec["name"],
        windows=len(data["candidates"]),
        positives=n_pos,
    )
print(
    f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in {time.monotonic() -
t0:.0f}s "
    f"| {len(data['candidates'])} windows ({n_pos} positive) | {len(data['gts'])} golden
rallies "
    f"-> {out}",
    flush=True,
)
written.append(out)
return written

```

```

def main() -> None:
    ap = argparse.ArgumentParser(
        description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
    )
    ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
    ap.add_argument("--out-dir", default="output")
    ap.add_argument("--frame-skip", type=int, default=3)
    ap.add_argument("--iou", type=float, default=0.5)
    ap.add_argument(
        "--fresh", action="store_true", help="re-extract even if a JSON already exists"
    )
    ap.add_argument(
        "--smallest-first",
        action="store_true",
        help="process smaller videos first for quick feedback (LOVO is order-independent)",
    )
    ap.add_argument(
        "--source-manifest",
        default=None,
        help=(
            "collector golden_source_videos.json; overrides video paths for source/proxy "
            f"videos (or set {SOURCE_MANIFEST_ENV})"
        ),
    )
    ap.add_argument(
        "--only",
        action="append",
        default=[],
        metavar="VIDEO_ID",
        help="process one manifest video by name; repeatable",
    )
    ap.add_argument(
        "--missing-fusion-only",
        action="store_true",
        help="skip videos whose existing JSON already has full shuttle+person fusion schema",
    )
    ap.add_argument(
        "--cpu",
        action="store_true",
        help="run torchvision person detection on CPU instead of CUDA",
    )
    ap.add_argument(
        "--detector-model",
        default=None,
        choices=[
            "fasterrcnn_resnet50_fpn",
            "fasterrcnn_mobilenet_v3_large_fpn",
            "retinanet_resnet50_fpn",
        ],
        help="torchvision detector model; default keeps the configured Faster R-CNN ResNet",
    )
    ap.add_argument(
        "--max-gpu-temp-c",
        type=float,
        default=None,
        help="abort extraction if telemetry sees GPU temperature at or above this value",
    )
    args = ap.parse_args()
    run(
        args.manifest,
        args.out_dir,
        args.frame_skip,
        args.iou,
        fresh=args.fresh,
        smallest_first=args.smallest_first,
    )

```

```

        source_manifest=args.source_manifest,
        only=args.only,
        missing_fusion_only=args.missing_fusion_only,
        use_gpu=not args.cpu,
        detector_model=args.detector_model,
        max_gpu_temp_c=args.max_gpu_temp_c,
    )

if __name__ == "__main__":
    main()

```

11. assistant (2026-07-04T00:44:38.037Z)

Now let me check the OLD version from master and compare the resume logic:

12. user (2026-07-04T00:44:39.642Z)

```

def run(
    manifest_path: str,
    out_dir: str,
    frame_skip: int = 3,
    iou: float = 0.5,
    fresh: bool = False,
    smallest_first: bool = False,
) -> List[str]:
    specs = load_resolved_manifest(manifest_path)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
        # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
        row/frame).
        specs = sorted(
            specs,
            key=lambda s: (
                os.path.getsize(s["trajectory"])
                if os.path.isfile(s.get("trajectory", ""))
                else 0
            ),
        )
    os.makedirs(out_dir, exist_ok=True)
    # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so keep a tiny
    # rotating runlog of velocity + machine health for the post-mortem. Constructing it must
    never
    # break extraction, so guard the build and fall back to no telemetry on any error.
    telem: Optional[RunTelemetry]
    try:
        telem = RunTelemetry(
            os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
        )
    except Exception: # noqa: BLE001 - telemetry wiring must never break the run it observes
        telem = None

```

13. user (2026-07-04T00:44:42.415Z)

```

for vi, spec in enumerate(specs, 1):
    out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
    # Resumable: skip a video whose JSON already exists (a re-run picks up where a prior
    # run left off instead of re-detecting). --fresh forces a redo.
    if not fresh and os.path.isfile(out):
        print(
            f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted "
            f"-> {out} (use --fresh to redo)",
            flush=True,
        )

```

```

        written.append(out)
        continue
    print(
        f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
        flush=True,
    )
    t0 = time.monotonic()
    if telem is not None:
        telem.event("video_start", spec["name"])
    try:
        data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
    except Exception as e: # noqa: BLE001 - record machine state at death, then re-raise
        if telem is not None:
            telem.event("error", spec["name"], err=str(e))
        raise
    n_pos = sum(c["label"] for c in data["candidates"])
    with open(out, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
    if telem is not None:
        telem.event(
            "video_done",
            spec["name"],
            windows=len(data["candidates"]),
            positives=n_pos,
        )
    print(
        f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in {time.monotonic() -
t0:.0f}s "
        f"| {len(data['candidates'])} windows ({n_pos} positive) | {len(data['gts'])} golden
rallies "
        f"-> {out}",
        flush=True,
    )
    written.append(out)
    return written

def main() -> None:
    ap = argparse.ArgumentParser(
        description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
    )
    ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
    ap.add_argument("--out-dir", default="output")

```

14. assistant (2026-07-04T00:44:45.110Z)

Now let me check the test files to understand angle 2:

15. user (2026-07-04T00:44:47.367Z)

```

def _golden_present() -> bool:
    """True iff >=6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the shuttle/person
cue
columns). Counts only ablatable files ? leaner velocity-windowing files are skipped by
`load_videos`, so counting them would trip the litmus on an un-ablatable corpus (the n=11
mixed-schema bug)."""
    n = 0
    for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
        try:
            with open(p, encoding="utf-8") as f:
                cand = json.load(f).get("candidates") or []
        except (OSError, ValueError):
            continue
    if _is_fusion_schema(
        cand
    )

```

```

        ): # canonical predicate (SSOT in backend.eval.experiment, #215)
            n += 1
    return n >= 6

@pytest.mark.skipif(
    not _golden_present(),
    reason="golden output/*_golden_features.json absent (run fusion_golden first)",
)
def test_litmus_reproduces_gate_a():
    """LITMUS: on real golden feature JSONs the runner reproduces the recorded Gate-A result.

    The legacy n=6 owner-box substrate preserves the 2026-06-14 hand measurement. After A7, the
    refreshed owner-box substrate has n=15 full-fusion files; that result is intentionally
    weaker
    and supports "keep structural, do not promote served default".
    """
    videos = ablation.load_videos(GOLDEN_DIR)
    assert len(videos) in {6, 15}
    cfg = ablation.litmus_config()
    report = run_ablation(videos, cfg)
    assert len(report.rows) == 1
    row = report.rows[0]
    assert row.signal == "gate_a"
    if len(videos) == 6:
        # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
        assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
        assert row.sign_wins == 6 and row.sign_losses == 0
    --
    not _golden_present(), reason="golden output/*_golden_features.json absent"
)
def test_litmus_report_is_well_formed():
    """The litmus report is a proper AblationReport (provenance + a sorted single row)."""
    report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
    assert isinstance(report, AblationReport)
    assert report.num_videos in {6, 15}
    assert report.full_set_signals == ["gate_a"]
    assert report.label_set_hash
    out = ablation.format_table(report)
    assert "gate_a" in out

```

----- edge cases

```

def test_min_players_gate_knob_zeroed():
    """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""

```

16. user (2026-07-04T00:44:50.191Z)

```

def test_litmus_reproduces_gate_a():
    """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-measured Gate-A
    result ? marginal ? F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
    tolerance.

    This validates the whole framework against the recorded ground truth. Skips cleanly in CI
    where the golden substrate is absent; runs on the owner box where it is present."""
    videos = ablation.load_videos(GOLDEN_DIR)
    assert len(videos) == 6
    cfg = ablation.litmus_config()
    report = run_ablation(videos, cfg)
    assert len(report.rows) == 1
    row = report.rows[0]
    assert row.signal == "gate_a"
    # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.

```

```

assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
assert row.sign_wins == 6 and row.sign_losses == 0
assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
assert row.ci_low == pytest.approx(0.042, abs=0.01)
assert row.ci_high == pytest.approx(0.104, abs=0.01)
assert row.ci_excludes_zero is True
# Gate-A is structural ? reported but never auto-promoted to earns_keep.
assert row.verdict == ablation.STRUCTURAL_PROTECTED

```

17. user (2026-07-04T00:44:52.678Z)

```

@pytest.mark.skipif(
    not _golden_present(),
    reason="golden output/*_golden_features.json absent (run fusion_golden first)",
)
def test_litmus_reproduces_gate_a():

```

18. assistant (2026-07-04T00:44:54.499Z)

Now let me check load_videos in ablation.py:

19. user (2026-07-04T00:44:55.830Z)

```

def load_videos(features_dir: str) -> List[VideoCandidates]:
    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
    (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
    (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0 so
    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.

    Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
    R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads the
    ablatable subset cleanly (see ``_is_fusion_schema``)."""
    return experiment.load_golden_videos(
        features_dir,
        shuttle_columns=SHUTTLE_COLUMNS,
        person_columns=PERSON_COLUMNS,
        audio_columns=AUDIO_COLUMNS,
        audio_policy="tolerate-missing",
        skip_logger=logger,
        skip_message="ablation.load_videos: skipping %s ? windowing-only schema "
        "(no shuttle/person cue dicts), not an ablatable fusion-feature file",
    )

def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
    """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3) weighted
    by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the learned path
    (distinct from the static Gate-A litmus)."""
    cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
    return experiment.Variant(
        name="full_learned",
        feature_names=cols,
        threshold=threshold,
        min_crossings=min_crossings,
    )

```

20. user (2026-07-04T00:44:58.432Z)

```

def load_videos(features_dir: str) -> List[VideoCandidates]:
    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
    (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
    (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0 so
    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.

    Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the

```



```

R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads the
ablatable subset cleanly (see ``_is_fusion_schema``)."""
return experiment.load_golden_videos(
    features_dir,
    shuttle_columns=SHUTTLE_COLUMNS,
    person_columns=PERSON_COLUMNS,
    audio_columns=AUDIO_COLUMNS,
    audio_policy="tolerate-missing",
    skip_logger=logger,
    skip_message="ablation.load_videos: skipping %s ? windowing-only schema "
    "(no shuttle/person cue dicts), not an ablatable fusion-feature file",
)

```

21. assistant (2026-07-04T00:45:00.942Z)

Now check the test_fusion_compare and test_audio_features for the video_path and MP4 guard:

22. user (2026-07-04T00:45:05.284Z)

```

    "name": "t",
    "trajectory": "bar.csv",
    "fps": 30.0,
    "frame_width": 1920.0,
    "labels": "foo.rallies.csv",
    "video_path": __file__,
}
data = fusion_golden.extract_video(
    spec, detector=_StubDetector(), frame_skip=3, iou=0.5
)

assert data["name"] == "t"
assert data["gts"] == [[0.0, 10.0]]
assert len(data["candidates"]) == 2
c0 = data["candidates"][0]
assert set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
--
    "name": "hot",
    "trajectory": "bar.csv",
    "fps": 30.0,
    "frame_width": 1920.0,
    "labels": "foo.rallies.csv",
    "video_path": __file__,
}

with pytest.raises(RuntimeError, match="GPU temperature 90.0C"):
    fusion_golden.extract_video(
        spec,
        detector=_StubDetector(),
        telemetry=_HotTelemetry(),
        max_gpu_temp_c=85.0,
    )
--
assert fusion_golden.is_full_fusion_feature_file(
    os.path.join(out_dir, "alpha_golden_features.json")
)

def test_fusion_golden_source_manifest_supplies_video_paths(tmp_path, monkeypatch):
    """The collector source manifest can provide Drive/F-drive video paths for A7 extraction."""
    from backend.eval import fusion_golden

    out_dir = str(tmp_path / "out")
    os.makedirs(out_dir)
    for name in ("a.csv", "b.csv", "a.rallies.csv", "b.rallies.csv"):

```

```

        (tmp_path / name).write_text("x\n", encoding="utf-8")
source_video = tmp_path / "source-alpha.mp4"
source_video.write_text("video\n", encoding="utf-8")
source_manifest = tmp_path / "golden_source_videos.json"
--
        "name": "alpha",
        "trajectory": "a.csv",
        "fps": 30.0,
        "frame_width": 1920.0,
        "labels": "a.rallies.csv",
        "video_path": str(tmp_path / "stale-alpha.mp4"),
    },
    {
        "name": "beta",
        "trajectory": "b.csv",
        "fps": 30.0,
        "frame_width": 1920.0,
        "labels": "b.rallies.csv",
    },
]
manifest_path = str(tmp_path / "m.json")
--
    json.dump(manifest, f)

seen = []

def _stub(spec, **kw):
    seen.append((spec["name"], spec.get("video_path")))
    return {
        "name": spec["name"],
        "fps": 30.0,
        "frame_width": 1920.0,
        "candidates": [],
        "gts": [],
    }

monkeypatch.setattr(fusion_golden, "extract_video", _stub)

--
order.clear()
fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)
assert order == ["big", "small"] # manifest order preserved

def test_derive_video_path():
    from backend.eval.fusion_golden import _derive_video_path

    assert (
        _derive_video_path({"labels": "X/Adarsh and Avi.rallies.csv"})
        == "X/Adarsh and Avi.MP4"
    )
    assert (
        _derive_video_path({"video_path": "a.mp4", "labels": "b.rallies.csv"})
        == "a.mp4"
    )
    with pytest.raises(ValueError):
        _derive_video_path({"name": "n", "labels": "weird.csv"})

```

23. user (2026-07-04T00:45:08.906Z)

```

order.clear()
fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)
assert order == ["big", "small"] # manifest order preserved

```

```

def test_derive_video_path():
    from backend.eval.fusion_golden import _derive_video_path

    assert (
        _derive_video_path({"labels": "X/Adarsh and Avi.rallies.csv"})
        == "X/Adarsh and Avi.MP4"
    )
    assert (
        _derive_video_path({"video_path": "a.mp4", "labels": "b.rallies.csv"})
        == "a.mp4"
    )
    with pytest.raises(ValueError):
        _derive_video_path({"name": "n", "labels": "weird.csv"})

```

24. user (2026-07-04T00:45:11.629Z)

```

def test_fusion_compare_needs_two_videos(tmp_path):
    _write_video(str(tmp_path), "only")
    videos = fusion_compare.load_videos(str(tmp_path))
    assert len(videos) == 1 # compare() / experiment.compare needs >=2 for honest LOVO

```

----- extractor assembly

```

def test_extract_video_assemblies_shuttle_and_person(monkeypatch):

```

25. user (2026-07-04T00:45:17.905Z)

```

def test_extract_video_assemblies_shuttle_and_person(monkeypatch):
    from backend.eval import fusion_golden
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    intervals = [(0.0, 10.0), (40.0, 41.0)]
    x_shuttle = [[10.0, 0.3, 0.2, 0.5, 4.0], [1.0, 0.1, 0.05, 0.2, 0.0]]
    points = [
        SimpleNamespace(frame=i, visible=True, x=500.0, y=(100.0 if i % 2 else 900.0))
        for i in range(10)
    ]

    monkeypatch.setattr(
        fusion_golden.distill_local,
        "build_candidates",
        lambda traj, fps, fw: (intervals, x_shuttle),
    )
    monkeypatch.setattr(
        TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda traj: points)
    )
    monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])

```

26. user (2026-07-04T00:45:20.459Z)

```

monkeypatch.setattr(
    fusion_golden.distill_local,
    "build_candidates",
    lambda traj, fps, fw: (intervals, x_shuttle),
)
monkeypatch.setattr(
    TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda traj: points)
)
monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])

class _StubDetector:
    def detections_over_windows(self, video, windows, frame_skip, progress_cb=None):

```

```

        return [
            {
                "frames": {sf: [(1, (10.0, 10.0, 30.0, 40.0))]},
                "fps": 30.0,
                "frame_width": 1920.0,
                "frame_height": 1080.0,
            }
            for (sf, ef) in windows
        ]

spec = {
    "name": "t",
    "trajectory": "bar.csv",
    "fps": 30.0,
    "frame_width": 1920.0,
    "labels": "foo.rallies.csv",
}
data = fusion_golden.extract_video(

```

27. assistant (2026-07-04T00:45:22.328Z)

Now let me check the test_audio_features file in the new version:

28. user (2026-07-04T00:45:23.423Z)

```

    os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8"
) as f:
    json.dump(data, f)

def test_augment_rebases_stale_manifest_before_deriving_video_path(tmp_path, monkeypatch):
    workspace = tmp_path / "workspace"
    repo = workspace / "badminton-highlight-indexer"
    out = repo / "output"
    traj = workspace / "Annotation Setup" / "Trajectories" / "foo.csv"
    labels = workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
    traj.parent.mkdir(parents=True)
    labels.parent.mkdir(parents=True)
    out.mkdir(parents=True)
    traj.write_text("x\n", encoding="utf-8")
    labels.write_text("start_time,end_time\n0,1\n", encoding="utf-8")
    labels.with_name("Foo.MP4").write_text("video\n", encoding="utf-8")
    manifest = out / "human_lovo_manifest.json"
    manifest.write_text(
        json.dumps(
            [
--
        assert seen == [(str(labels)[-len(".rallies.csv")] + ".MP4", 1.5)]
        data = json.loads(feature_json.read_text(encoding="utf-8"))
        assert data["candidates"][0]["audio"]["audio_hit_count"] == 1.0

def test_augment_uses_source_manifest_video_path(tmp_path, monkeypatch):
    workspace = tmp_path / "workspace"
    repo = workspace / "badminton-highlight-indexer"
    out = repo / "output"
    traj = workspace / "Annotation Setup" / "Trajectories" / "foo.csv"
    labels = workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
    source_video = workspace / "source" / "Foo_proxy.mp4"
    traj.parent.mkdir(parents=True)
    labels.parent.mkdir(parents=True)
    source_video.parent.mkdir(parents=True)
    out.mkdir(parents=True)
    traj.write_text("x\n", encoding="utf-8")
    labels.write_text("start_time,end_time\n0,1\n", encoding="utf-8")

```

```

source_video.write_text("video\n", encoding="utf-8")
manifest = out / "human_lovo_manifest.json"
manifest.write_text(

```

29. user (2026-07-04T00:45:26.172Z)

```

    os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8"
) as f:
    json.dump(data, f)

```

```

def test_augment_rebases_stale_manifest_before_deriving_video_path(tmp_path, monkeypatch):
    workspace = tmp_path / "workspace"
    repo = workspace / "badminton-highlight-indexer"
    out = repo / "output"
    traj = workspace / "Annotation Setup" / "Trajectories" / "foo.csv"
    labels = workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
    traj.parent.mkdir(parents=True)
    labels.parent.mkdir(parents=True)
    out.mkdir(parents=True)
    traj.write_text("x\n", encoding="utf-8")
    labels.write_text("start_time,end_time\n0,1\n", encoding="utf-8")
    manifest = out / "human_lovo_manifest.json"
    manifest.write_text(
        json.dumps(
            [
                {

```

30. assistant (2026-07-04T00:45:28.126Z)

Let me check what happens in the actual new test file for video_path requirements:

31. user (2026-07-04T00:45:29.354Z)

```

"""Tests for the P4 audio cue ? pure audio features + the incremental-lift driver (no
GPU/ffmpeg)."""

```

```

import json
import os
from types import SimpleNamespace

```

```

import pytest

```

```

from backend.eval import fusion_audio
from backend.pipeline.detectors import audio_features as af

```

```

def H(t, strength=1.0):
    return SimpleNamespace(t=t, strength=strength)

```

```

def test_compute_audio_features_empty():
    f = af.compute_audio_features([], 0.0, 10.0)
    assert set(f) == set(af.AUDIO_FEATURE_NAMES)
    assert all(v == 0.0 for v in f.values())

```

```

def test_compute_audio_features_counts_rate_strength():
    hits = [
        H(1.0, 2.0),
        H(2.0, 4.0),
        H(5.0, 6.0),
        H(50.0, 1.0),
    ] # 3 in [0,10), 1 outside

```

```

f = af.compute_audio_features(hits, 0.0, 10.0)
assert f["audio_hit_count"] == 3.0
assert f["audio_hit_rate"] == pytest.approx(0.3) # 3 hits / 10 s
assert f["audio_hit_strength_mean"] == pytest.approx((2 + 4 + 6) / 3)

def test_compute_audio_features_half_open_boundary():
    hits = [H(10.0), H(20.0)] # window [10,20): 10 in, 20 out
    f = af.compute_audio_features(hits, 10.0, 20.0)
    assert f["audio_hit_count"] == 1.0

```

----- incremental-lift driver (synthetic)

```

def _write_video(d, name):
    """shuttle + person constant (no signal); audio_hit_count separates rallies (8) from junk
    (0)."""
    cand = []
    for (s, e), pos in [
        ((0.0, 10.0), 1),
        ((20.0, 30.0), 1),
        ((40.0, 41.0), 0),
        ((50.0, 51.0), 0),
    ]:
        shuttle = {fn: 1.0 for fn in fusion_audio.SHUTTLE_FEATURE_NAMES}
        person = {fn: 1.0 for fn in fusion_audio.PERSON_FEATURE_NAMES}
        audio = {fn: 0.0 for fn in af.AUDIO_FEATURE_NAMES}
        audio["audio_hit_count"] = 8.0 if pos else 0.0
        cand.append(
            {
                "start": s,
                "end": e,
                "shuttle": shuttle,
                "person": person,
                "audio": audio,
                "label": pos,
            }
        )
    data = {
        "name": name,
        "fps": 30.0,
        "frame_width": 1920.0,
        "candidates": cand,
        "gts": [[0.0, 10.0], [20.0, 30.0]],
    }
    with open(
        os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8"
    ) as f:
        json.dump(data, f)

```

32. user (2026-07-04T00:45:32.137Z)

```

def test_augment_rebases_stale_manifest_before_deriving_video_path(tmp_path, monkeypatch):
    workspace = tmp_path / "workspace"
    repo = workspace / "badminton-highlight-indexer"
    out = repo / "output"
    traj = workspace / "Annotation Setup" / "Trajectories" / "foo.csv"
    labels = workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
    traj.parent.mkdir(parents=True)
    labels.parent.mkdir(parents=True)
    out.mkdir(parents=True)
    traj.write_text("x\n", encoding="utf-8")
    labels.write_text("start_time,end_time\n0,1\n", encoding="utf-8")

```

```

labels.with_name("Foo.MP4").write_text("video\n", encoding="utf-8")
manifest = out / "human_lovo_manifest.json"
manifest.write_text(
    json.dumps(
        [
            {
                "name": "foo",
                "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
                "labels": "C:/old/Annotation Setup/Collect/Golden Labelled/Foo.rallies.csv",
                "fps": 30.0,
                "frame_width": 1920.0,
            }
        ]
    ),
    encoding="utf-8",
)
feature_json = out / "foo_golden_features.json"
feature_json.write_text(
    json.dumps(
        {
            "name": "foo",
            "fps": 30.0,
            "frame_width": 1920.0,
            "candidates": [{"start": 0.0, "end": 1.0}],
            "gts": [[0.0, 1.0]],
        }
    ),
    encoding="utf-8",
)
seen = []

def _fake_detect(video_path, k=2.5):
    seen.append((video_path, k))
    return [H(0.5, 3.0)]

monkeypatch.setenv("RALLY_GOLDEN_SEARCH_ROOTS", str(workspace))
monkeypatch.setattr(fusion_audio, "detect_hits_in_video", _fake_detect)

written = fusion_audio.augment(str(manifest), str(out), k=1.5)

```

33. assistant (2026-07-04T00:45:34.245Z)

Now let me check if there's a check in extract_video or augment for os.path.isfile(video):

34. user (2026-07-04T00:45:35.535Z)

(Bash completed with no output)

35. user (2026-07-04T00:45:38.053Z)

```

video_path = _derive_video_path(spec)

if detector is None:
    from backend.pipeline.detectors.person_detector import PersonDetector

    detector = PersonDetector({"indexing": {"use_gpu": True}})

```

36. assistant (2026-07-04T00:45:40.040Z)

Now let me check the docs for the A8 command mentioned in angle 4:

37. user (2026-07-04T00:45:41.546Z)

The corpus is now large enough to move from "grow labels first" into alpha-readiness work, but

the 15-video headline is not itself a launch gate pass. `[verified]` The golden manifest has 15 badminton videos; `[analysis]` A2-A7 made the local manifest/path and full-fusion/audio feature surface reproducible on all 15; `[analysis]` A4/A5 refreshed the n=15 nightly and heuristic floors; `[analysis]` A8/A9 still reject served Gate A default promotion and owned-model serving claims on the current evidence.

The first controlled alpha should be a hosted, authenticated product loop: upload a video, process it, select rallies, compile, and download a reel. The owned-model/commercial rally-detection claim remains a separate NO-GO until the ship-gate target is ratified and the WASB checkpoint provenance is resolved.

1. Problem & goal

The previous launch blockers were spread across roadmap docs and mixed together three different questions:

- **Can an invited tester use the product without SSH?**
- **Can the 15-video golden corpus now support honest quality gates?**
- **Can we claim or ship an owned/commercial rally detector?**

This doc separates those into executable workstreams. The goal is to reach an invite-only alpha where the product is usable end-to-end, the quality claims are honest and reproducible, and the remaining model/legal gates are explicit rather than accidentally implied by launch activity.

2. Decisions locked

```
| # | Decision | Source / date | Implication |
|---|---|---|---|
| D1 | Product alpha and owned-model launch are separate decisions. | `NEXT_STEPS.md` NO-GO banner, 2026-06-30; verified 2026-07-03 | We can launch a controlled hosted product loop without claiming the owned model is commercially cleared. |
| D2 | First alpha must be page-driven, not SSH/CLI-driven. | `NEXT_STEPS.md` P0 UI-driven/shareable item; verified 2026-07-03 | Hosted service, edge auth, upload, job polling, compile, and download are launch-critical. |
| D3 | The 15-video corpus is now a measurement asset, not a free launch pass. | `[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md`; `[analysis]` local eval runs, 2026-07-03 | Rebase/portability, feature completeness, baseline refresh, and gate ratification come before quality claims. |
--
- `[analysis]` A7 generated full shuttle/person fusion schema for all 15 videos and A8 audio augmentation populated audio features for all 15.
- `[analysis]` `python -m backend.eval.fusion_compare --features-dir output` now runs over all 15 and measures person-fusion `Delta F1 = +0.008`; the paired CI crosses zero, so person fusion is not promotable by itself.
- `[analysis]` `python -m backend.eval.fusion_audio --features-dir output --compare` measures audio `Delta F1 = +0.018`; the paired CI crosses zero, so audio is a shadow/suggestive cue, not a default-flip.
- `[analysis]` `python -m backend.eval.serve_contrast --manifest output\human_lovo_manifest.json` measured proposal Gate A mean `Delta F1 +0.0287`, but served mean `Delta F1 -0.0328`; not served-safe.
- `[analysis]` `python -m training.gen0.harness --manifest output\human_lovo_manifest.json --floor eval_baselines\heuristic_lovo_n15.json --version 0.1.1-n15-shadow.20260704 --out artifacts\rally_tal_gen0` produced learned LOVO mean F1 `0.551`, floor passed versus heuristic `0.446`, but sign test `9/15`, `p = 0.3036`, `ship=False`.
- `[analysis]` `python scripts/nightly_regression.py --manifest <rebased-n15> --features-dir output --no-report` reported default/milestone tolF1 `0.3636` vs baseline-off tolF1 `0.1382`, but marked the frozen baseline stale due to config/corpus changes.
- `[analysis]` Before A6, `python -m backend.eval.ablation --features-dir output --granularity group` failed at import due to a circular import between `backend/eval/ablation.py` and `backend/eval/ablation_pdf.py`; A6 fixes the module entrypoint and keeps full-fusion ablation scoped to the 6 videos with full schema until A7.
```

4. Launch strategy

4.1 Alpha lane: make the product usable

The alpha lane is successful when an invited tester can open the app, authenticate, upload a video, watch the job progress, pick rallies, compile a reel, and download it. This lane should avoid model-quality promises beyond the measured state and should default to the most reliable serving path.

4.2 Quality lane: make the 15-video corpus gate-ready

The quality lane is successful when the 15-video corpus is portable, reproducible, feature-complete, and backed by refreshed baselines. Only then should R2 toLF1, Gen-0/TemporalMaxer, or other model/promotion decisions be ratified.

4.3 Flywheel lane: make tester feedback useful

The flywheel lane is successful when user corrections can flow into the collector/vault path with consent/provenance and a visible re-eval delta. This is valuable for alpha learning, but it should not block the first controlled alpha if manual promotion can bridge the gap.

```
--
| A8 | Re-run `fusion_compare`, group ablation, and served contrast after A7; explicitly
promote/reject person fusion, Gate A served-default, and any other R-series default flips |
`badminton-highlight-indexer` | A6, A7 | Reviewer sign-off | IN PROGRESS | this PR |
| A9 | Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor; produce a non-
serving artifact bundle and gate verdict | `badminton-highlight-indexer` | A5, A8 | Owner
approval for M0.4 scope | IN PROGRESS | this PR |
| A10 | Decide and record the alpha ship-gate target: metric, corpus slice, minimum threshold,
significance rule, and what claims are allowed | Owner + `badminton-highlight-indexer` | A4, A5,
A8 | Owner decision | GATED | - |
| A11 | Resolve or explicitly defer WASB C3 provenance for alpha messaging; if deferred,
document "no commercial model sharing" copy guardrail | Owner/legal | A10 | Legal/provenance |
GATED | - |
| A12 | Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector promotion,
`train`/`reeval` job, delta-F1 surface, and alignment with `COMPUTE_DECOUPLED_SERVING` M11 |
`khealsutra` + `badminton-highlight-indexer` + collector | A1, A3 | Owner D13 reopen
(`khealsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`) | GATED | - |
| A13 | Alpha launch review: runbook, privacy/consent copy, observability/job-failure triage,
known limits, rollback/disable steps, and owner GO/NO-GO | Cross-repo | A1, A4, A10, A11 | Owner
launch decision | TODO | - |
```

8. Concrete next PR sequence

1. ****Docs PR:**** land this tracker.
2. ****Corpus portability PR:**** rebase/portable manifest tooling plus missing-file smoke test.
3. ****Hosted alpha PR/batch:**** health/auth/upload/process/job/compile/download verification and runbook updates.
4. ****Baseline PR:**** intentional n=15 nightly baseline refresh and heuristic floor refresh.
5. ****Ablation/fusion PRs:**** fix ablation import, complete 15-video features, rerun promotion/rejection decisions.
6. ****Launch decision PR/docs update:**** record ship-gate target, C3 status, alpha copy limits, and launch checklist.

A2 smoke command:

```
```powershell
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
--write output/human_lovo_manifest.local.json
```
```

--

A8 fusion/default-decision record:

```
```powershell
python -m backend.eval.fusion_compare --features-dir output
```

```
python -m backend.eval.fusion_audio --features-dir output --compare
python -m backend.eval.ablation --features-dir output --granularity group
python -m backend.eval.serve_contrast --manifest output\human_lovo_manifest.json
python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json --features-dir
output --preset served --max-window-duration 0 --inactivity-end 0 --vs served --vs-max-window-
duration 6 --vs-inactivity-end 1.5 --json-out output\A6_r2_ablation.json
```

```

Result:

- Person fusion is **not promoted**: shuttle-only F1 `0.302`, shuttle+person F1 `0.311`, `Delta F1 +0.008`, paired CI `[-0.040, +0.055]`, sign `8W/6L`, `p = 0.791`.
- Audio is **shadow/suggestive, not promoted**: shuttle+person F1 `0.311`, shuttle+person+audio F1 `0.329`, `Delta F1 +0.018`, paired CI `[-0.017, +0.053]`, sign `10W/4L`, `p = 0.180`.
- Group ablation remains non-significant for cue promotion: audio `+0.018` CI `[-0.017, +0.053]`; shuttle `+0.013` CI `[-0.002, +0.029]`; gate_a `-0.001` CI `[-0.012, +0.008]`; person `-0.009` CI `[-0.045, +0.021]`.
- Gate A served default is **rejected**: proposal mean `Delta F1 +0.0287`, CI `[-0.0222, +0.0680]`; served mean `Delta F1 -0.0328`, CI `[-0.0799, +0.0052]`, `regresses=True`.
- R2/cap/inactivity remains the strong quality result: served candidate F1 `0.472` vs baseline-off `0.237`, `Delta F1 +0.236`, CI `[+0.161, +0.313]`; tolF1 `0.364` vs `0.138`, `Delta tolF1 +0.225`, CI `[+0.165, +0.283]`; merge rate `0.135` vs `0.638`. Net over-seg guard passes, strict per-video split-count guard still blocks blanket promotion language.

A9 Gen-0 shadow-training record:

```
```powershell
--
- 2026-07-04 - Recorded A7 full-fusion/audio regeneration, A8 n15 fusion/default decisions, and
A9 Gen-0 n15 shadow-training gate verdict.
```

### 38. user (2026-07-04T00:45:45.315Z)

```
| A8 | Re-run `fusion_compare`, group ablation, and served contrast after A7; explicitly
promote/reject person fusion, Gate A served-default, and any other R-series default flips |
`badminton-highlight-indexer` | A6, A7 | Reviewer sign-off | TODO | - |
| A9 | Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor; produce a non-
serving artifact bundle and gate verdict | `badminton-highlight-indexer` | A5, A8 | Owner
approval for M0.4 scope | TODO | - |
| A10 | Decide and record the alpha ship-gate target: metric, corpus slice, minimum threshold,
significance rule, and what claims are allowed | Owner + `badminton-highlight-indexer` | A4, A5,
A8 | Owner decision | GATED | - |
| A11 | Resolve or explicitly defer WASB C3 provenance for alpha messaging; if deferred,
document "no commercial model sharing" copy guardrail | Owner/legal | A10 | Legal/provenance |
GATED | - |
| A12 | Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector promotion,
`train`/reeval` job, delta-F1 surface, and alignment with `COMPUTE_DECOUPLED_SERVING` M11 |
`khelsutra` + `badminton-highlight-indexer` + collector | A1, A3 | Owner D13 reopen
(`khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`) | GATED | - |
| A13 | Alpha launch review: runbook, privacy/consent copy, observability/job-failure triage,
known limits, rollback/disable steps, and owner GO/NO-GO | Cross-repo | A1, A4, A10, A11 | Owner
launch decision | TODO | - |
```

## 8. Concrete next PR sequence

1. **Docs PR**: land this tracker.
2. **Corpus portability PR**: rebase/portable manifest tooling plus missing-file smoke test.
3. **Hosted alpha PR/batch**: health/auth/upload/process/job/compile/download verification and runbook updates.
4. **Baseline PR**: intentional n=15 nightly baseline refresh and heuristic floor refresh.
5. **Ablation/fusion PRs**: fix ablation import, complete 15-video features, rerun promotion/rejection decisions.
6. **Launch decision PR/docs update**: record ship-gate target, C3 status, alpha copy limits, and launch checklist.

A2 smoke command:

```
```powershell
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
--write output/human_lovo_manifest.local.json
```
```

The checker resolves stale absolute paths against the repo's sibling `Annotation Setup` folders and `output/`, then exits non-zero if any required `trajectory` or `labels` file is absent.

A4 nightly baseline refresh record:

```
```powershell
python scripts\nightly_regression.py --manifest output\human_lovo_manifest.json --features-dir
output --update-baseline
```
```

Result:

### 39. assistant (2026-07-04T00:45:47.241Z)

Perfect. Now let me search for uses of `extract_video` and `augment` in non-test code:

### 40. user (2026-07-04T00:45:48.638Z)

```
backend/eval/ablation.py: (older JSONs predate the audio augment) ? a missing block defaults
those columns to 0.0 so
backend/eval/experiment.py: ``fusion_audio.augment`` and older JSONs predate it).
backend/eval/experiment.py: ``no_audio_error`` formatted with ``path``), mirroring
"run ``--augment`` first".
backend/eval/fusion_audio.py:Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-
detect): `augment` runs
backend/eval/fusion_audio.py: python -m backend.eval.fusion_audio --manifest
output/human_lovo_manifest.json --features-dir output --augment --compare
backend/eval/fusion_audio.py:def augment(
backend/eval/fusion_audio.py: f"{spec.get('name')}: source video not found for
audio augmentation: {video}"
backend/eval/fusion_audio.py: columns (requires `augment` to have run ? raises if a fusion
JSON lacks audio).
backend/eval/fusion_audio.py: hasn't been ``--augment``-ed still raises rather than being
silently dropped.""
backend/eval/fusion_audio.py: no_audio_error="{path} has no audio features ? run
``--augment`` first",
backend/eval/fusion_audio.py: help="augment one manifest video by name; repeatable",
backend/eval/fusion_audio.py: "--augment",
backend/eval/fusion_audio.py: if args.augment:
backend/eval/fusion_audio.py: augment(
backend/eval/fusion_audio.py: raise SystemExit(f"need >=2 augmented feature files in
{args.features_dir}")
backend/eval/fusion_golden.py:def extract_video(
backend/eval/fusion_golden.py: data = extract_video(
backend/eval/gemini_refine.py:sports.badminton prompt (sport_registry),
utils.video.extract_video_chunk,
backend/eval/gemini_refine.py:from backend.utils.video import extract_video_chunk,
get_video_duration
backend/eval/gemini_refine.py: if not extract_video_chunk(
backend/eval/gemini_refine.py: if not extract_video_chunk(
backend/eval/rally_seg_eval.py: tIoU here is the SECONDARY trend-line per the augment-then-
ablation-gate decision): strict-1:1
backend/eval/tolerance_metrics.py:rally-end honest ? IAA-calibrated later); augment tIoU now,
ablation-gate any eventual swap.
backend/pipeline/segmenters/gemini.py:from backend.utils.video import extract_video_chunk,
get_video_duration
backend/pipeline/segmenters/gemini.py: success = extract_video_chunk(
```

```

backend/pipeline/segmenters/trajectory_hybrid.py:from backend.utils.video import
extract_video_chunk, get_video_duration
backend/pipeline/segmenters/trajectory_hybrid.py: if not extract_video_chunk(
backend/pipeline/segmenters/yolo_hybrid.py:from backend.utils.video import extract_video_chunk,
get_video_duration
backend/pipeline/segmenters/yolo_hybrid.py: success =
extract_video_chunk(video_path, cs, actual_dur, chunk_path)
backend/pipeline/segmenters/yolo_hybrid.py: success = extract_video_chunk(

```

#### 41. assistant (2026-07-04T00:45:50.889Z)

Now let me check what the new test files do with video\_path:

#### 42. user (2026-07-04T00:45:52.339Z)

```

+++ b/tests/test_fusion_compare.py
@@ -181,6 +181,7 @@ def test_extract_video_assembles_shuttle_and_person(monkeypatch):
 "fps": 30.0,
 "frame_width": 1920.0,
 "labels": "foo.rallies.csv",
+ "video_path": __file__,
 }
 data = fusion_golden.extract_video(
 spec, detector=_StubDetector(), frame_skip=3, iou=0.5
@@ -197,6 +198,47 @@ def test_extract_video_assembles_shuttle_and_person(monkeypatch):
 assert all(isinstance(v, float) for v in c0["person"].values())
--
+ "name": "hot",
+ "trajectory": "bar.csv",
+ "fps": 30.0,
+ "frame_width": 1920.0,
+ "labels": "foo.rallies.csv",
+ "video_path": __file__,
+ }
+
+ with pytest.raises(RuntimeError, match="GPU temperature 90.0C"):
+ fusion_golden.extract_video(
+ spec,
--
+ assert fusion_golden.is_full_fusion_feature_file(
+ os.path.join(out_dir, "alpha_golden_features.json")
+)
+
+
+def test_fusion_golden_source_manifest_supplies_video_paths(tmp_path, monkeypatch):
+ """The collector source manifest can provide Drive/F-drive video paths for A7
+ extraction."""
+ from backend.eval import fusion_golden
+
+ out_dir = str(tmp_path / "out")
+ os.makedirs(out_dir)
--
+ "name": "alpha",
+ "trajectory": "a.csv",
+ "fps": 30.0,
+ "frame_width": 1920.0,
+ "labels": "a.rallies.csv",
+ "video_path": str(tmp_path / "stale-alpha.mp4"),
+ },
+ {
+ "name": "beta",
+ "trajectory": "b.csv",
+ "fps": 30.0,
--
+ json.dump(manifest, f)

```

```

+
+ seen = []
+
+ def _stub(spec, **kw):
+ seen.append((spec["name"], spec.get("video_path")))
+ return {
+ "name": spec["name"],
+ "fps": 30.0,
+ "frame_width": 1920.0,
+ "candidates": [],

```

#### 43. assistant (2026-07-04T00:45:54.593Z)

Now let me get the full PR diff to understand the complete set of changes:

#### 44. user (2026-07-04T00:45:56.928Z)

(Bash completed with no output)

#### 45. user (2026-07-04T00:45:59.773Z)

```

diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index aee6d7e..0666a0c 100644
--- a/backend/eval/fusion_golden.py
+++ b/backend/eval/fusion_golden.py
@@ -21,7 +21,7 @@ import argparse
 import json
 import os
 import time
-from typing import Any, Dict, List, Optional
+from typing import Any, Dict, List, Optional, Sequence

 from backend.eval import distill_local
 from backend.eval.calibrate_wasb import load_golden_gts
@@ -36,6 +36,100 @@ from backend.utils.run_telemetry import RunTelemetry
SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
+SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
+
+
+def is_full_fusion_feature_file(path: str) -> bool:
+ """True when an existing golden feature JSON already has shuttle+person cue dicts.
+
+ Corpus-growth runs can leave leaner windowing-only JSONs with the same filename pattern.
+ Those are useful for gts/windowing analysis, but they are not the replayable fusion
+ substrate that ``fusion_compare`` and ``ablation`` need.
+ """
+
+ try:
+ with open(path, encoding="utf-8") as f:
+ data = json.load(f)
+ except (OSError, json.JSONDecodeError, TypeError):
+ return False
+ candidates = data.get("candidates")
+ if not isinstance(candidates, list):
+ return False
+ if not candidates:
+ return True
+ return all(
+ isinstance(candidate, dict)
+ and "shuttle" in candidate
+ and "person" in candidate
+ for candidate in candidates

```

```

+)
+
+
+def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
+ """Load ``video_id -> local_path`` from the collector golden-source manifest.
+
+ The collector records source/proxy ownership and full-file MD5s. Only entries marked
+ ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
+ """
+
+ if not source_manifest_path:
+ return {}
+ with open(source_manifest_path, encoding="utf-8") as f:
+ data = json.load(f)
+ entries = data.get("videos", data) if isinstance(data, dict) else data
+ if not isinstance(entries, list):
+ raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
+
+ paths: Dict[str, str] = {}
+ for entry in entries:
+ if not isinstance(entry, dict):
+ continue
+ video_id = entry.get("video_id") or entry.get("name")
+ local_path = entry.get("local_path")
+ if not video_id or not local_path:
+ continue
+ if entry.get("source_status", "present_local") != "present_local":
+ continue
+ paths[str(video_id)] = str(local_path)
+ return paths
+
+def attach_source_video_paths(
+ specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
+) -> int:
+ """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
+
+ Returns the number of specs updated. The collector source manifest is authoritative
+ for local source/proxy video location and overrides derived/stale manifest paths.
+ """
+
+ source_paths = load_source_video_paths(source_manifest_path)
+ updated = 0
+ for spec in specs:
+ name = str(spec.get("name", ""))
+ source_path = source_paths.get(name)
+ if not source_path:
+ continue
+ if spec.get("video_path") == source_path:
+ continue
+ spec["video_path"] = source_path
+ updated += 1
+ return updated
+
+def _filter_specs(
+ specs: Sequence[Dict[str, Any]], only: Sequence[str]
+) -> List[Dict[str, Any]]:
+ if not only:
+ return list(specs)
+ wanted = list(dict.fromkeys(only))
+ wanted_set = set(wanted)
+ filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
+ found = {str(spec.get("name")) for spec in filtered}
+ missing = [name for name in wanted if name not in found]

```

```

+ if missing:
+ raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
+ return filtered

def _derive_video_path(spec: Dict[str, Any]) -> str:
@@ -58,6 +152,9 @@ def extract_video(
 iou: float = 0.5,
 log_every_sec: float = 15.0,
 telemetry: Optional[Any] = None,
+ use_gpu: bool = True,
+ detector_model: Optional[str] = None,
+ max_gpu_temp_c: Optional[float] = None,
) -> Dict[str, Any]:
 """One golden video -> a feature record (shuttle + person features per candidate window).

@@ -77,11 +174,18 @@ def extract_video(
 # (parity with the shuttle net-crossing gate) rather than hard-coding an orientation.
 axis, net_px, _span = rally_gate.estimate_play_axis(points)
 video_path = _derive_video_path(spec)
+ if not os.path.isfile(video_path):
+ raise FileNotFoundError(
+ f"{spec.get('name')}: source video not found for fusion extraction: {video_path}"
+)

 if detector is None:
 from backend.pipeline.detectors.person_detector import PersonDetector

- detector = PersonDetector({"indexing": {"use_gpu": True}})
+ indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
+ if detector_model:
+ indexing_cfg["detector_model"] = detector_model
+ detector = PersonDetector({"indexing": indexing_cfg})

 # Single-pass person detection over ALL candidate windows (open the video ONCE, read
 # forward ? no per-window seek). Then compute features per window (pure, no GPU).
@@ -95,8 +199,17 @@ def extract_video(
 state = {"last": t0}

 def _progress(done: int, total: int) -> None:
+ rec = None
+ if telemetry is not None:

```

#### 46. user (2026-07-04T00:46:03.527Z)

```

 for vi, spec in enumerate(specs, 1):
 out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
- # Resumable: skip a video whose JSON already exists (a re-run picks up where a prior
- # run left off instead of re-detecting). --fresh forces a redo.
- if not fresh and os.path.isfile(out):
+ existing = os.path.isfile(out)
+ existing_full = existing and is_full_fusion_feature_file(out)
+ # Resumable: skip a video whose full-fusion JSON already exists. A leaner
+ # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
+ if missing_fusion_only and not fresh and existing_full:
+ print(
+ f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
+ f"features already present -> {out}",
+ flush=True,
+)
+ written.append(out)
+ continue
+ if not fresh and existing and existing_full:
+ print(
+ f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted "

```

```

 f"-> {out} (use --fresh to redo)",
@@ -197,6 +335,12 @@ def run(
)
 written.append(out)
 continue
+ if not fresh and existing and not existing_full:
+ print(
+ f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
+ "is not full-fusion schema; re-extracting...",
+ flush=True,
+)
 print(
 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
 flush=True,
@@ -205,7 +349,15 @@ def run(
 if telem is not None:
 telem.event("video_start", spec["name"])
 try:
- data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
+ data = extract_video(
+ spec,
+ frame_skip=frame_skip,
+ iou=iou,
+ telemetry=telem,
+ use_gpu=use_gpu,
+ detector_model=detector_model,
+ max_gpu_temp_c=max_gpu_temp_c,
+)
 except Exception as e: # noqa: BLE001 - record machine state at death, then re-raise
 if telem is not None:
 telem.event("error", spec["name"], err=str(e))
@@ -246,6 +398,47 @@ def main() -> None:
 action="store_true",
 help="process smaller videos first for quick feedback (LOVO is order-independent)",
)
+ ap.add_argument(
+ "--source-manifest",
+ default=None,
+ help=(
+ "collector golden_source_videos.json; overrides video paths for source/proxy "
+ f"videos (or set {SOURCE_MANIFEST_ENV})"
+)
+)

```

#### 47. assistant (2026-07-04T00:46:05.543Z)

Perfect! Now let me check the complete `run` function signature change:

#### 48. user (2026-07-04T00:46:06.551Z)

(Bash completed with no output)

#### 49. user (2026-07-04T00:46:09.563Z)

```

def run(
 manifest_path: str,
 out_dir: str,
 frame_skip: int = 3,
 iou: float = 0.5,
 fresh: bool = False,
 smallest_first: bool = False,
) -> List[str]:
 specs = load_resolved_manifest(manifest_path)
 if smallest_first:
 # Process small videos first for quick feedback (the slow full match lands last);
 # the LOVO result is order-independent. Proxy size by the trajectory CSV (one

```



```
row/frame).
 specs = sorted(
```

#### 50. assistant (2026-07-04T00:46:11.550Z)

Let me get more of the run function in the new version:

#### 51. user (2026-07-04T00:46:12.683Z)

```
def run(
 manifest_path: str,
 out_dir: str,
 frame_skip: int = 3,
 iou: float = 0.5,
 fresh: bool = False,
 smallest_first: bool = False,
 source_manifest: Optional[str] = None,
 only: Sequence[str] = (),
 missing_fusion_only: bool = False,
 use_gpu: bool = True,
 detector_model: Optional[str] = None,
 max_gpu_temp_c: Optional[float] = None,
) -> List[str]:
 specs = load_resolved_manifest(manifest_path)
 source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
 if source_manifest:
 updated = attach_source_video_paths(specs, source_manifest)
 print(
 f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
 f"from {source_manifest}",
```

#### 52. assistant (2026-07-04T00:46:15.608Z)

Now let me look at the test changes more carefully for angle 3:

#### 53. user (2026-07-04T00:46:16.793Z)

```
+ progress_cb(1, 10)
+ return []
+
+ spec = {
+ "name": "hot",
+ "trajectory": "bar.csv",
+ "fps": 30.0,
+ "frame_width": 1920.0,
+ "labels": "foo.rallies.csv",
+ "video_path": __file__,
+ }
+
+
```

#### 54. assistant (2026-07-04T00:46:18.748Z)

Let me check if there are any places where extract\_video is called from non-test code without a video\_path in the spec:

#### 55. user (2026-07-04T00:46:19.666Z)

```
backend/eval/fusion_golden.py: def extract_video(
backend/eval/fusion_golden.py: data = extract_video(
```

#### 56. assistant (2026-07-04T00:46:22.661Z)

Good, extract\_video is only called from within fusion\_golden.py. Now let me check the A8 command with max-window-duration and inactivity-end in both versions:

## 57. user (2026-07-04T00:46:23.691Z)

A6 ablation/R2 record:

```
```powershell
python -m backend.eval.ablation --features-dir output --granularity group
python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json --features-dir
output --preset served --max-window-duration 0 --inactivity-end 0 --vs served --vs-max-window-
duration 6 --vs-inactivity-end 1.5 --json-out output\a6_r2_ablation.json
```
```

Result: the ablation CLI now runs, but full-fusion feature ablation still reports `n=6` because only the original six feature JSONs carry the full fusion schema; A7 must regenerate/upconvert the newer nine before fusion/person/audio promotion decisions. The n=15 R2/tolF1 contrast is positive for the served milestone versus baseline-off: baseline/off `tolF1 = 0.138`, served `tolF1 = 0.364`, `Delta tolF1 = +0.225` with CI `[+0.165, +0.283]`, sign `14/0`, `p = 0.000`; `Delta F1@0.5 = +0.236` with CI `[+0.161, +0.313]`; merge rate improves from `0.638` to `0.135`. Net over-segmentation guard passes (`1.276 -> 0.378`), while the strict segment-count rule remains blocked by increased splits, so R2/tolF1 is the supported gate for the baseline-off to served milestone contrast rather than a blanket fusion/default-flip approval.

## 9. Open questions

1. **\*\*Owner:\*\*** Is the first alpha allowed to use manual/offline corpus promotion, or should P10/P11 be reopened before any tester wave?
2. **\*\*Owner/platform:\*\*** Which hosted compute target is the next stand-up: Cloud Run GPU, GCE L4 VM, or another approved GPU host?
3. **\*\*Owner/legal:\*\*** What is the acceptable alpha wording while WASB C3 remains unresolved?
4. **\*\*Engineering:\*\*** Should the portable corpus manifest live as a generated artifact, a checked-in redacted manifest, or both?

```
--
python -m backend.eval.fusion_compare --features-dir output
python -m backend.eval.serve_contrast --manifest <rebased-n15-manifest>
python -m training.gen0.harness --manifest <rebased-n15-manifest> --floor
eval_baselines\heuristic_lovo_n15.json --version analysis-n15 --out <temp-out>
python scripts\nightly_regression.py --manifest <rebased-n15-manifest> --features-dir output
--no-report
python -m backend.eval.ablation --features-dir output --granularity group
python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json --features-dir
output --preset served --max-window-duration 0 --inactivity-end 0 --vs served --vs-max-window-
duration 6 --vs-inactivity-end 1.5 --json-out output\a6_r2_ablation.json
```
```

Changelog

- 2026-07-03 - Created from a synced five-repo review of `NEXT\_STEPS`, `ROADMAP`, golden-corpus docs, local eval artifacts, and the current 15-video corpus state.

58. assistant (2026-07-04T00:46:47.980Z)

Good! Now let me double-check the critical resume logic around full-fusion in the new code:

Based on my analysis of the PR diff and comparing old vs new code, here are the 6 key invariant violations to flag:

```
```json
[
 {
 "file": "backend/eval/fusion_golden.py",
 "line": 300,
 "summary": "Resume guard split: NEW code with --missing-fusion-only flag skips FULL-fusion
files but re-extracts LEAN windowing-only JSONs. However, without --missing-fusion-only, both
```

branches now skip FULL-fusion files. The OLD code's simple 'file exists ? skip' applied to ALL file types. Risk: if --missing-fusion-only is NOT passed on a resume, the first check (line 300-309) is skipped and re-extracts full-fusion files (GPU waste). In default mode (no --missing-fusion-only), a full-fusion JSON DOES get skipped as intended (line 311-319), but the logic is fragile ? missing\_fusion\_only=False is the implicit default and depends on correct caller awareness.",

"failure\_scenario": "A user or CI runs `python -m backend.eval.fusion\_golden --manifest output/human\_lovo\_manifest.json --out-dir output` (no --missing-fusion-only flag) on a corpus with existing FULL-fusion JSONs. The second check (line 311-319) correctly skips them. But if a future change swaps the order of these checks or adds an early return, a FULL-fusion file could be re-extracted, wasting a GPU pass on a file that already has all needed schema."

```
},
{
 "file": "tests/test_ablation.py",
 "line": 47,
 "summary": "Test weakened from assert len(videos) == 6 to assert len(videos) in {6, 15}. The test's purpose is to guard the n=11 mixed-schema bug (mentioned in load_videos docstring). The old assertion rejected ANY corpus that wasn't exactly 6. The new assertion accepts both 6 and 15 BUT also silently accepts 11 (or any value >=6 and in the set). If a corpus load bug regressed and returned 11 videos (6 full-fusion + 5 windowing-only incorrectly marked as full), the test would NOT catch it ? it only checks set membership, not _is_fusion_schema correctness per video.",
```

"failure\_scenario": "A developer accidentally modifies load\_videos or \_is\_fusion\_schema to classify windowing-only JSONs as full-fusion. A corpus with 11 mixed videos loads and passes the litmus test (11 in {6,15} is False, so this specific case fails, but a corpus with 15 actual videos where 5 are misclassified would pass and produce wrong ablation results)."

```
},
{
 "file": "backend/eval/fusion_golden.py",
 "line": 177,
 "summary": "NEW guard added: os.path.isfile(video_path) check in extract_video. This guard did NOT exist in the OLD code. The old extract_video would fail downstream (in PersonDetector or detections_over_windows) with a less clear error. NEW code fails early with FileNotFoundError. RISK: all callers of extract_video (only fusion_golden.run()) must now provide a valid video_path. Pre-existing callers in non-test code must have a video file at the path, or the entire run() call fails. If a manifest entry has no video_path and derives to a stale/missing .MP4 path, the entire batch aborts.",
```

"failure\_scenario": "A caller runs fusion\_golden.run() on a manifest where one video's derived path (from labels/\*.rallies.csv -> \*.MP4) is stale or missing. The run aborts at that video with FileNotFoundError instead of skipping it (no resume-and-skip-missing logic exists). Compare to the old behavior: old code would also crash, but the error came from PersonDetector, so this is not a regression in BEHAVIOR, only in when the error fires."

```
},
{
 "file": "tests/test_fusion_compare.py",
 "line": 184,
 "summary": "NEW test requirement: spec dict now includes 'video_path': __file__. The old test had NO video_path; extract_video would derive it from labels (foo.rallies.csv -> foo.MP4). NEW code requires os.path.isfile(video_path) to be true BEFORE attempting extraction. The test now provides __file__ (the test module's path) as a fake video. This does NOT break the test (it passes because __file__ exists), but it masks the fact that the new guard depends on a valid file path. If a future test omits video_path and labels is not derivable to an existing file, the test will fail with FileNotFoundError.",
```

"failure\_scenario": "A new test calls extract\_video(spec, ...) with a spec that has no video\_path AND labels does not end with .rallies.csv (or the derived .MP4 does not exist). The test fails with FileNotFoundError. The old code would also fail, but further downstream (PersonDetector), giving a different error message and potentially allowing mocking/stubbing of PersonDetector to prevent the check."

```
},
{
 "file": "backend/eval/fusion_audio.py",
 "line": 66,
 "summary": "NEW guard added: os.path.isfile(video) check in augment(). Similar to fusion_golden's new check (line 177), this guard did NOT exist in the OLD code. Augment will now crash early with FileNotFoundError if the source video is missing, instead of failing in
```

```

detect_hits_in_video(). NEW code also calls attach_source_video_paths() and _filter_specs()
before the loop, which were NOT in the old code. These are new happy-path features, but they
also introduce new failure modes: if source_manifest cannot be loaded, attach_source_video_paths
will raise; if --only names a non-existent video, _filter_specs will raise with ValueError.",
 "failure_scenario": "A caller runs fusion_audio.augment() on a spec list where one video has
no local source video file (e.g., a proxy-only video marked present_local: false in the source
manifest). The run aborts at that video with FileNotFoundError instead of skipping or logging a
warning. The old code would also crash in detect_hits_in_video, but the error message would be
different (likely from ffmpeg or audio detector).",
 },
 {
 "file": "docs/ALPHA_LAUNCH_READINESS.md",
 "line": 105,
 "summary": "A8 command signature change: OLD A6 command uses `--max-window-duration 0
--inactivity-end 0`. NEW A8 command STILL uses the same flags and values. No apparent behavior
change documented. However, the NEW docs no longer mention A6 as a separate prior run ? A8 now
subsumes A6's role (rerun fusion_compare, ablation, served contrast). The command is identical,
so no behavior regression, but the context is now A7+A8 (new features) rather than A6 (old
features). Verify that the command still produces the intended R2/tolF1 gate metric with the new
15-video corpus.",
 "failure_scenario": "A user follows the old A6 workflow and re-runs the old command on a
corpus that now has 15 videos instead of 6. The served window parameters (cap=0, inactivity=0)
remain the same, but the corpus size changes the statistical power and CI. The gate's decision
(pass/fail on R2) might flip if n=15 yields a different tolF1 delta or CI. The docs do not warn
that A6 and A8 are different workflows; a user might assume --max-window-duration 0
--inactivity-end 0 produces the same QUALITY_ITERATIONS result on n=15 as it did on n=6 (it does
not).",
 }
]
...

```